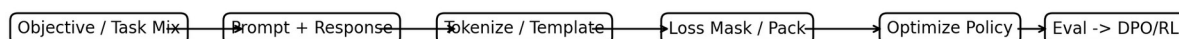


LLM 后训练之 SFT

Supervised Fine-Tuning 深度研究与工程手册

从 Behavior Prior、Loss Mask、Chat Template 与 Packing，到 Reasoning / Tool / Agent SFT、遗忘控制
和生产级 Recipe

SFT is a behavior compiler, not just “more pretraining”



Data semantics + serialization + token masking + optimization all decide which behavior gets probability mass.

研究日期：2026-08-13

系列：LLM Post-Training Engineering

目录与研究地图

模块	核心问题
0-3 第一性原理	SFT 到底在学什么？为什么不是“把知识灌进去”？目标函数与行为先验如何理解？
4-8 数据到梯度	Dataset schema、chat template、special token、loss mask、multi-turn target、packing 如何决定梯度？
9-14 数据与阶段	质量/多样性/难度、general/domain/reasoning/tool/agent SFT 如何分别组织？
15-19 优化与资源	Full FT、LoRA/QLoRA、batch/LR、warmup/scheduler、precision、packing、分布式训练
20-23 稳定性	过拟合、灾难性遗忘、checkpoint 选择、train loss 与行为质量的错位
24-27 后训练接口	SFT -> DPO/RL、cold-start、distillation、production feedback loop
28-31 落地	生产级 SFT Contract、Recipes、失败模式、成熟度模型与上线检查单
32 参考资料	2021-2026 关键论文与官方技术文档

阅读方式：如果你在搭训练平台：优先看第 4-8、15-19、28-31 章；如果你研究 reasoning/agent：优先看第 11-14、24-27 章；如果你负责数据：优先看第 6、9、10、21、28 章。

0. 执行摘要：SFT 是把“理想行为分布”写入 Policy 的第一层

Supervised Fine-Tuning (SFT) 的表面形式仍然是 next-token prediction，但训练数据已经从自然文本变成“状态 -> 理想动作”的行为示范。它的关键价值不是凭空增加世界知识，而是把预训练模型已有的能力重新组织成稳定、可调用、可评测的 assistant policy。

SFT is a behavior compiler, not just “more pretraining”



Data semantics + serialization + token masking + optimization all decide which behavior gets probability mass.

图 0-1 SFT 是从目标和示范到可用 Policy 的行为编译链。

层	SFT 实际决定什么	典型错误
Data Semantics	模型应该经历哪些任务与理想行为	只追求样本数量
Serialization	system/user/assistant/tool 如何变成 token 序列	训练与推理 chat template 不一致
Target Mask	哪些 token 真正获得梯度	把 user/system 当目标学；或错误 mask assistant
Optimization	改变 policy 的强度和速度	LR 太大导致漂移；epoch 太多过拟合
Evaluation	何时停止、是否保留通用能力	只看 train/val loss
Downstream Interface	是否为 DPO/RL 提供可探索的起点	SFT 太弱或太窄，后续优化困难

一句话： SFT 的核心不是“学习这个答案”，而是“提高在这种状态下生成这类动作的概率”。

1. 从 Pretraining 到 SFT：同一个 Loss，不同的学习语义

Pretraining: $x_1 \dots x_T \rightarrow$ predict every next token in natural corpus

SFT: context x + target $y \rightarrow$ predict the intended assistant behavior y

标准 causal SFT 仍然最小化 token-level negative log-likelihood:

$$L_{\text{SFT}}(\theta) = - \sum_t m_t * \log p_{\theta}(y_t | x, y_{<t})$$

其中 m_t 是 loss mask。真正区分 SFT 与普通 continued pretraining 的，不只是数据来源，而是“哪些 token 被视为目标”、对话结构、任务分布和目标行为的一致性。

公开研究证据：InstructGPT 先用标注员编写的理想 demonstrations 对 GPT-3 做 supervised fine-tuning，再进入 preference/RLHF；这把 SFT 明确放在“建立行为起点”的位置。[1]

工程推论：因此在 Agent 语境中，可以把 SFT 看成 behavior cloning：给定 state/context，模仿 expert action/trajectory。后续 DPO/RL 再在这个先验附近做选择或搜索。

2. SFT 真正在学什么：Behavior Prior、Format Prior、Routing Prior

Prior	模型学到的东西	例子
Behavior Prior	面对某类任务应该采取什么动作	问答、拒绝、澄清、总结、执行
Format Prior	输出结构与停止方式	JSON、代码块、工具调用、EOT
Routing Prior	何时切换不同策略	什么时候用工具、什么时候直接回答
Reasoning Prior	复杂任务常见的解题轨迹	分解、验证、反思、计算
Interaction Prior	多轮对话中的角色和状态	引用历史、承接 tool result
Safety/Policy Prior	边界与合规行为	安全完成、拒绝或重定向

公开研究证据： LIMA 用仅 1,000 条精心挑选的数据训练 65B LLaMA，并提出“Superficial Alignment Hypothesis”：预训练学得大部分知识和能力，而少量高质量 alignment data 主要教交互格式与行为风格。[3]

风险提示： 不要把“多数知识来自预训练”误读成“领域 SFT 不能学知识”。SFT 可以改变事实/技能表现，但它的样本效率和泛化机制与大规模预训练不同，而且过强的窄域更新会带来遗忘与分布偏置。

3. SFT 与 Continued Pretraining / Domain Adaptation 的边界

方法	典型数据	Loss target	最适合解决
Continued Pretraining	领域原始文本 / code / docs	几乎所有 token	领域语言模型、知识暴露、分布适配
Instruction SFT	prompt -> ideal response	通常 response/assistant token	指令遵循、行为格式、任务策略
Domain SFT	领域任务 -> 专家回答	目标输出	工作流、术语、决策格式、场景行为
Preference Optimization	chosen > rejected	pairwise / implicit reward	哪种回答更好
RL / RLVR	rollout + reward	policy objective	探索与发现更优策略

Know-How: 如果目标是“模型没见过足够的领域语料”，优先考虑 continued pretraining；如果目标是“模型知道但不会按你的任务格式做”，SFT 更直接；两者可以组合，但要分别评测知识与行为。

4. SFT 数据契约：不要让 Dataset 只剩 text 字段

```
{
  "sample_id": "...",
  "stage": "sft",
  "task": "tool_use",
  "difficulty": "medium",
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."},
    {"role": "assistant", "tool_calls": [...]},
    {"role": "tool", "content": "..."},
    {"role": "assistant", "content": "..."}
  ],
  "target_spans": [...],
  "source": "human|synthetic|distill",
  "verifier": {"type": "unit_test", "score": 1.0},
  "quality_score": 0.94,
  "dataset_version": "sft-v12"
}
```

字段	为什么重要
task / capability	支持 mixture、slice eval、failure mining
source / generator	区分 human / synthetic / distilled 行为偏差
target_spans	明确哪些 token 应该贡献 loss
verifier / quality	追踪 “为什么认为这是好 demonstration”
template_version	避免训练和推理 serialization 漂移
dataset_version / lineage	支持复现实验、回滚、污染审计

工程推论：生产级 SFT 数据集应该是可审计的数据资产，而不是一个不可解释的 JSONL。

5. Chat Template：训练和推理之间最隐蔽的接口契约

模型看到的并不是 role 字段，而是模板序列化后的 token。system/user/assistant/tool 标签、BOS/EOS/EOT、换行和 generation prefix 都会改变条件分布。

messages -> Jinja/chat template -> special tokens -> token IDs -> attention/mask -> loss

风险	现象	后果
Train/Inference mismatch	训练用 A 模板，推理用 B 模板	明显能力下降、停止异常、角色混淆
EOT/EOS 混淆	把 turn end 和 sequence end 混为一谈	多轮生成提前结束或串台
Special token 未注册	模板引用 tokenizer 不认识的 token	embedding 未训练或 token 被拆分
Tool serialization 漂移	tool call JSON/marker 与部署时不同	tool-use SFT 失效
Generation prefix 错误	prompt 末尾未正确指向 assistant	模型续写 user/system

公开研究证据： LIMA 为 user/assistant utterance 引入专用 EOT token，以区分 turn boundary 与已有 EOS 语义。[3]
Hugging Face TRL 当前 SFTTrainer 也把 chat template、EOS、assistant mask 作为显式训练配置。[12]

6. Loss Mask：SFT 最重要但常被忽视的“梯度路由器”

Assistant-only / completion-only masking



The model sees the whole conversation as context, but gradients are applied only to selected target tokens.

图 6-1 assistant-only masking：上下文参与前向条件化，但不一定参与监督损失。

策略	Loss 位置	优点	风险
Full-sequence LM	system+user+assistant 全部	实现最简单；保留 LM 风格	会学习复述 prompt/role token；目标语义混杂
Completion-only	prompt=0, completion=1	适合 prompt-completion	多轮中 target 定义仍可能含糊
Assistant-only	所有 assistant span=1	最符合 chat behavior cloning	依赖模板正确生成 assistant mask
Selective span	特定回答/工具/最终答案=1	可控制行为部件	实现复杂；容易 mask bug
Weighted token/turn	不同 span 不同权重	可强调 final/tool/reasoning	权重设计需要 ablation

公开研究证据： Hugging Face TRL SFTTrainer 当前支持 `completion_only_loss`、`assistant_only_loss` 和 `packing`；`assistant-only` 依赖 chat template 能提供 assistant token mask。[12]

风险提示： Mask bug 通常不会导致训练报错。Loss 仍会下降，但模型学错对象。因此必须对随机 batch 解码并可视化 target mask。

7. Multi-turn SFT：到底监督每一轮 Assistant，还是只监督最后一轮？

方案	适合场景	优点	风险
All assistant turns	真实多轮对话	每轮都有监督，样本效率高	早期 assistant 可能只是上下文，不是目标策略
Last-turn only	多轮 context -> final response	目标清晰	浪费前序 expert action 信号
Turn-weighted	Agent / tool trace	可强调关键 action/final	需要可靠标注和训练支持
Trajectory-level curated	长链 agent	保留完整 state-action 分布	单条错误会污染后续状态

Know-How：对 tool/agent SFT，最先确定的不是 LR，而是：哪几轮是 expert action？哪些只是 environment observation？哪些 token 应被 mask？

8. Packing、Padding-Free 与 Attention Isolation：吞吐优化不能改变训练语义

机制	目的	必须检查
Padding	对齐 batch 长度	padding token 不贡献 loss；attention mask 正确
Packing	多个短样本塞进固定长度序列	样本边界、position ids、loss mask 不串
Padding-free	把 batch flatten 减少 padding	attention kernel/position 逻辑支持
Sequence truncation	控制 max length	不能把答案关键尾部或 tool result 截掉
Length bucketing	相似长度分 batch	保持 shuffle 与 task mixture

公开研究证据：TRL 当前将 packing、packing strategy、padding-free、max_length、truncation 等作为 SFTConfig 的显式工程选项。[12]

工程推论：训练吞吐指标应该看“有效 target tokens/s”，而不是 raw tokens/s。一个 batch 里大量 prompt/padding token 若不参与 loss，GPU 利用率高不等于学习信号密度高。

9. Data Quantity vs Quality vs Diversity：SFT 最经典的三角问题

维度	太低	太高/失控	目标
Quality	错误示范、风格噪声	过度人工同质化	可靠而非单一
Diversity	只会模板内任务	长尾质量下降	覆盖能力空间
Quantity	行为先验不稳	重复/低信息数据淹没有效梯度	在边际收益下降前停止
Difficulty	太简单无学习信号	超出能力形成噪声	Learnable Frontier

公开研究证据： LIMA 的 ablation 指出：单纯增加数据量出现显著边际递减，而 prompt diversity 和数据质量带来更大收益；它用 1,000 个高质量示范得到强行为对齐。[3] FLAN 则从另一侧说明多任务覆盖规模能够显著增强 unseen-task generalization。[2]

公开研究证据： Super-NaturalInstructions 包含 1,616 个任务、76 类任务，并系统研究“训练任务数、每任务样本数、模型规模”对跨任务泛化的影响。[15]

10. SFT Mixture：不是把所有高质量数据直接 concat

控制轴	例子	为什么需要
Capability	chat / reasoning / code / tool / safety	避免单一能力挤占梯度
Difficulty	easy / frontier / hard	控制 curriculum 与学习信号
Source	human / synthetic / distill / real-user	控制风格与 teacher bias
Length	short / medium / long	控制 token 预算和上下文能力
Language	zh/en/multi	防止高资源语言压制低资源语言
Risk	normal / policy-sensitive	保证行为边界覆盖

Token Mixture != Example Mixture: 1 条 8K-token reasoning trajectory 与 1 条 200-token QA 对 GPU 和梯度贡献完全不同。SFT recipe 应至少同时跟踪 example share、target-token share、total-token share。

11. General SFT：建立可用 Assistant 的基础行为层

目标	示范数据应覆盖	不应该只靠
Instruction Following	多约束、多格式、多语言、拒绝/澄清	简单问答
Interaction Style	简洁/结构/上下文承接	统一模板腔
Open-domain QA	知识调用与不确定性表达	benchmark 题库
Writing / Transformation	总结、改写、分类、抽取、创作	一种输出长度
Conversation	多轮追问、纠错、引用上下文	单轮 prompt

公开研究证据：InstructGPT 的 SFT demonstrations 来自标注员编写的理想行为，并与真实 API prompt 分布结合。[1]
Self-Instruct、WizardLM 等进一步展示了 synthetic instruction generation / evolution 对扩大任务覆盖和复杂度的作用。
[13][16]

12. Reasoning SFT：教“轨迹先验”，而不是把 CoT 当神圣文本

数据形态	价值	主要风险
Problem -> verified reasoning -> answer	建立可读的解题轨迹	teacher reasoning 可能有隐藏错误
Problem -> concise rationale -> answer	降低 token 成本	复杂任务监督不足
Multiple valid trajectories	增加策略多样性	数据生成/验证成本高
Error-correction trajectory	训练恢复与自检	可能学会形式化反思而非真正修正

公开研究证据：FLAN 的研究把 chain-of-thought data 纳入 instruction fine-tuning，并报告对多类模型/benchmark 的提升。[2] DeepSeek-R1 则显示纯 RL 的 R1-Zero 存在可读性和语言混杂问题，因此 R1 引入 cold-start data 和多阶段训练，SFT 在这里承担“可读且稳定的 reasoning prior”角色。[7]

风险提示：Reasoning SFT 的 correctness 应尽可能由 answer/verifier 保证。漂亮的长 CoT 不等于正确；训练错误推理比训练短答案更可能把错误模式写入 policy。

13. Tool-Use SFT：训练的是“何时调用、调用什么、参数怎么填、结果怎么用”

SFT target shape changes with the behavior being taught

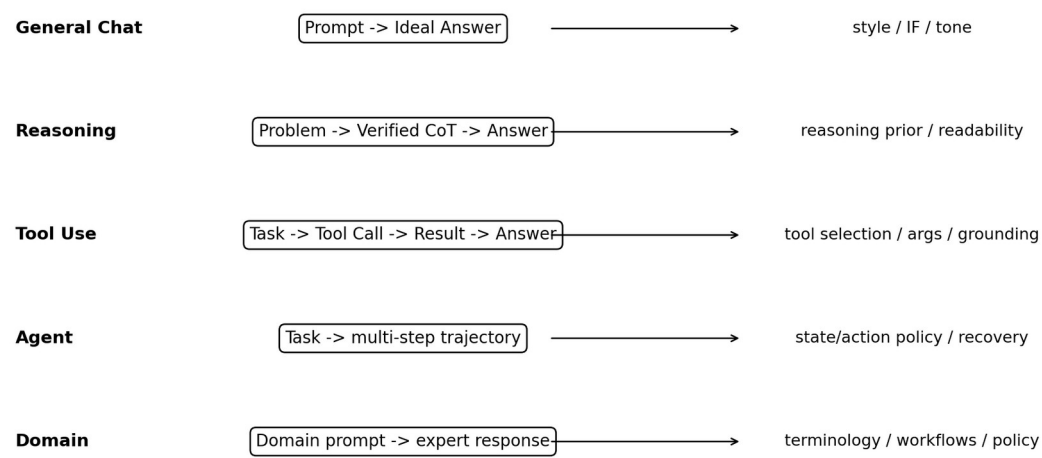


图 13-1 不同 SFT 场景的 target 结构不同。

子能力	SFT 目标	Verifier
Tool selection	直接回答 vs tool A/B/C	任务 label / execution success
Arguments	合法且语义正确的 JSON/参数	schema + semantic checks
Observation grounding	基于 tool result 而非臆测	result consistency
Retry / recovery	失败后修改参数或换工具	environment success
Stop decision	完成后不继续无谓调用	task completion + cost

公开研究证据： Llama 3 将 tool usage 作为其后训练能力之一，并在 post-training 中结合高质量示范与偏好/选择数据。
[4] AutoIF 展示了把 instruction-following 约束转化为可执行验证，再用 execution feedback 做 rejection sampling，从而生成可用于 SFT 和 preference training 的数据。[14]

14. Agent SFT：从 Response 模仿升级为 Trajectory Behavior Cloning

Task -> State_0 -> Action_0 -> Observation_1 -> Action_1 -> ... -> Final

Agent SFT 难点	为什么比 Chat SFT 难	工程处理
State distribution shift	学生早期动作不同，后续状态不再属于 expert 数据	加入 recovery/perturbed trajectories；后续 RL
Credit assignment	一条轨迹最终成功，不代表每步都优	step annotation / verifier / selective loss
Long horizon	错误累积	分段 curriculum + checkpoint tasks
Tool cost	示范可能过度调用工具	在示范中保留高效策略
Environment nondeterminism	相同动作结果不同	记录环境版本与 seed

工程推论：Agent SFT 更接近 imitation learning。它能快速形成可用 action prior，但无法完全解决 distribution shift；这也是为什么长程 Agent 往往最终还需要在线 rollout、recovery data 或 RL。

15. Domain SFT：如何增强领域行为而不“训窄”

策略	作用	风险
Domain-only SFT	快速强化专域任务	通用能力/风格遗忘
General + Domain Mix	保留基础助手能力	需要调 mixture
Replay / Anchor Set	显式保留旧能力	额外 token 成本
PEFT Adapter	隔离一部分域适配	容量有限；adapter 管理复杂
Continued PT + SFT	先补语料再教行为	训练成本更高

公开研究证据：持续 instruction fine-tuning 的实证研究观察到 1B-7B 模型在 domain knowledge、reasoning 和 reading comprehension 上存在 catastrophic forgetting，并发现 general instruction tuning 可缓解后续持续微调的遗忘。[11]

Know-How：上线前必须做“目标域提升”和“非目标域回归”双矩阵。SFT 不是单指标优化：领域准确率 +5 但通用指令遵循 -12，通常不是成功。

16. Full Fine-Tuning vs LoRA vs QLoRA：选择的是“可塑性 - 成本 - 运维”平衡

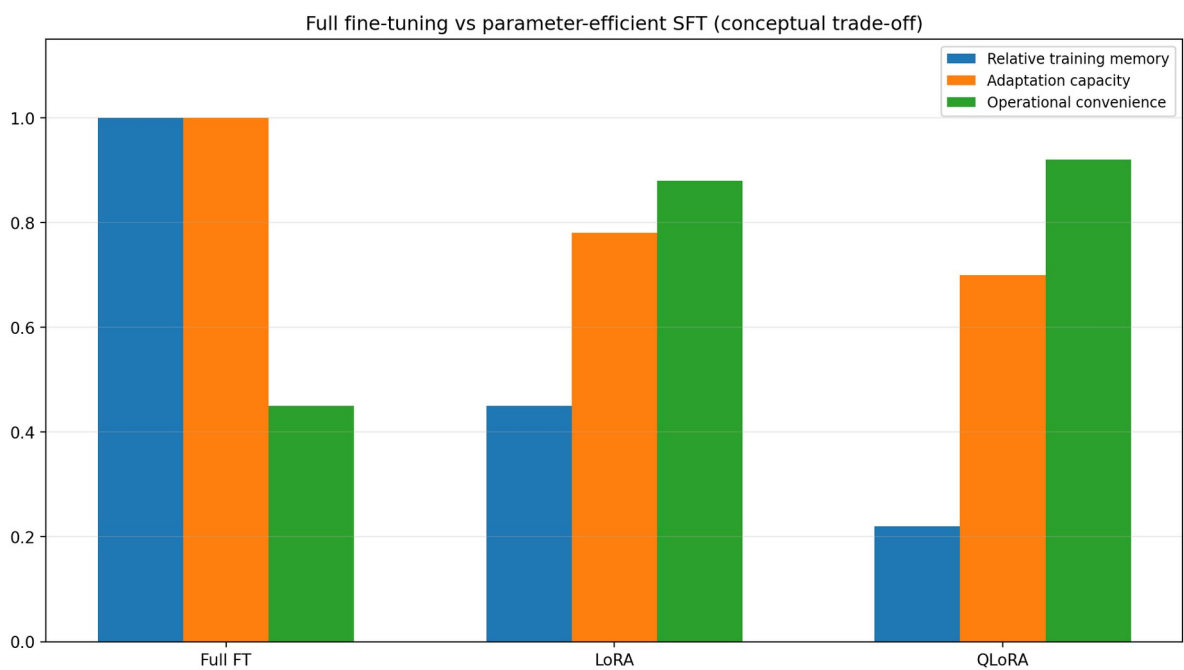


图 16-1 Full FT / LoRA / QLoRA 的概念性权衡；实际结果依模型、数据与实现而变。

方案	参数更新	优点	主要限制
Full FT	全部参数	最大可塑性；统一 checkpoint	显存/优化器状态昂贵；易整体漂移
LoRA	低秩 adapter	参数少、训练/存储方便、可多 adapter	容量和 target modules 选择影响效果
QLoRA	4-bit frozen base + LoRA	大幅降低显存门槛	量化/内核/adapter 配置更复杂

公开研究证据： LoRA 通过冻结预训练权重并注入低秩可训练矩阵，大幅减少可训练参数与显存；QLoRA 进一步在 4-bit quantized frozen base 上训练 LoRA，并展示了在单张 48GB GPU 上微调 65B 模型的可行性。[8][9]

风险提示： PEFT 不是“永远无损”。如果需要大幅重塑行为、工具协议或跨域能力，adapter 容量、rank、target modules 和数据规模必须实测。

17. Hyperparameters：SFT 没有跨模型通用的“神奇 LR”

参数	控制什么	实践建议
Learning Rate	每步参数移动幅度	从保守区间开始，按 model size / full vs PEFT / domain distance 搜索
Batch / Tokens per Update	梯度方差与更新频率	同时记录 example batch 与 target-token batch
Epochs / Steps	数据重复程度	小高质量集尤其容易多 epoch 过拟合
Warmup	早期更新稳定性	不要盲目复制 pretraining 比例
Scheduler	后期更新强度	cosine/linear/constant 需 dev ablation
Weight Decay	正则化	与 optimizer/adapter 设置共同调
Grad Clip	抑制异常 batch	监控触发频率而非只开开关

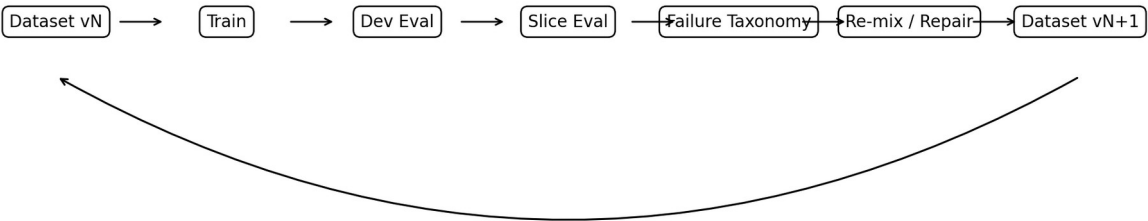
公开研究证据： LIMA 报告其 65B SFT 使用 1e-5 初始 LR、线性衰减、batch 32、15 epochs，并发现 validation perplexity 与生成质量并不一致，因此通过 held-out prompt 的行为质量选 checkpoint。[3]

公开研究证据： 2024 的系统 SFT recipe 研究在 3B-7B 多模型上发现：较大 batch 配合较低 LR 往往更好；早期 gradient norm/loss 对最终效果有预测价值；stacked 与 phased training 的表现差异不显著，而 stacked 更简单/样本效率更高。该结论是实验设置相关的，不应机械外推到所有模型。[6]

18. Optimization 监控：Loss 下降只是必要条件，不是成功定义

监控项	要回答的问题
Train NLL	模型是否在拟合 target token?
Dev NLL	是否开始过拟合相似分布?
Gradient Norm	更新是否异常、是否出现数据/数值尖峰?
Update/Weight Norm	模型偏离 base 多快?
Token Throughput	系统是否高效? 有效 target-token/s 是多少?
Slice Eval	行为真的变好吗? 哪类能力回归?
Generation Audit	格式、停止、冗长、模板腔是否出现?

Production SFT is an iterative control loop



Do not choose checkpoints by train loss alone; use behavior slices, regression tests and downstream readiness.

图 18-1 生产 SFT 应用行为评测闭环选择训练进度，而非只看 NLL。

19. Precision、Distributed、Memory：训练 Infra 要服务“实验可比性”

模块	常见选择	真正要控制的变量
Precision	bf16 / fp16 / fp32 master	数值稳定、吞吐、硬件支持
Sharding	FSDP / ZeRO	参数、梯度、optimizer state 的内存分配
Activation	checkpointing / offload	显存与计算交换
Attention	FlashAttention 等	长序列吞吐与 mask 正确性
Optimizer	AdamW / variants	state memory、稳定性、超参
Reproducibility	seed / data order / tokenizer hash	保证 recipe 对比不是随机波动

Know-How：训练系统升级后要保留 canonical small run。内核、packing 或 tokenizer 改动都可能改变训练语义；先验证 loss mask、batch token 数、样本顺序和 eval 一致，再宣称吞吐提升。

20. 过拟合：SFT 的典型问题不是“记住答案”，而是“学窄行为分布”

症状	原因	诊断
模板腔严重	输出风格过于一致	style slice / diverse prompts
过度解释	长回答占比过高	length-conditioned eval
任何问题都套同一流程	task diversity 不足	unseen task / OOD eval
拒绝率异常	安全示范比例/模板偏差	policy slice
工具滥用	tool demonstrations 过密	no-tool-needed slice
benchmark 提升但真实任务下降	训练/评测分布过近	held-out real-user set

公开研究证据： LIMA 发现 prompt diversity 比单纯扩数据量更关键，并发现 perplexity 不能可靠代表 generation quality。[3] 这直接说明小型高质量 SFT 的 checkpoint 选择必须依赖行为评估。

21. Catastrophic Forgetting 与 Alignment Tax：怎么知道 SFT 改坏了什么？

层	Regression Suite
Knowledge	MMLU/QA/domain-neutral factual slices
Reasoning	math / logic / code reasoning
In-context Learning	few-shot / format adaptation
Multilingual	非主训练语言
Long Context	长输入 retrieval / instruction following
Safety / Policy	边界行为、over-refusal
Base LM Quality	困惑度只作辅助，不作唯一判断

公开研究证据：持续 fine-tuning 的研究系统观察到 catastrophic forgetting，并指出模型在 fine-tuning distribution 上提升可能以其他能力为代价。[11]

工程推论：最有效的工程防线通常不是单一正则项，而是：general anchor/replay + mixture control + conservative LR + multi-slice eval + early stop。

22. Checkpoint Selection：为什么最低 Validation Loss 可能不是最好模型

选择信号	优点	局限
Val NLL	稳定、便宜	与开放生成质量可能错位
Benchmark score	可比较	容易污染/过拟合；覆盖窄
LLM-as-Judge	覆盖开放任务	judge bias / style bias
Deterministic Verifier	客观	只覆盖可验证任务
Human Eval	高保真	昂贵、慢
Composite Score	更符合产品目标	权重必须透明

公开研究证据： LIMA 明确报告 perplexity 与 generation quality 不相关，因此在第 5-10 epoch 间使用 held-out 50-example dev set 手工选择 checkpoint。[3]

推荐： 保存多个 checkpoint，先做 cheap automatic gates，再做中成本 judge/verifier，最后对 Pareto 候选做人工/真实任务验证。不要在训练过程中只保留“最低 loss 一个点”。

23. SFT Data Augmentation：何时 synthetic / noisy training 有价值？

方法	核心思想	适合解决
Self-Instruct	模型自生成 instruction/input/output 后过滤	扩大任务覆盖
Evol-Instruct	逐步提高 instruction 复杂度	复杂指令能力
Teacher Distillation	强模型生成高质量 response/trajectory	把强行为压给学生
Execution-filtered synthesis	通过 code/test 验证 response	可验证 instruction/tool/code
NEFTune	训练时给 embedding 加噪声	正则化/提升 instruction FT 泛化

公开研究证据： Self-Instruct 报告在 vanilla GPT-3 上用自生成并过滤的 instruction data 显著增强 instruction following。[16] WizardLM 用 Evol-Instruct 构造更复杂的 instruction。[13] AutoInstruct 用可执行 feedback 过滤 instruction-following 数据。[14]

公开研究证据： NEFTune 提出在 embedding 上加入噪声，并在多种 instruction datasets / models 上报告显著提升；其效果仍应在目标模型和数据上独立复现，而不应视为默认必开项。[10]

24. SFT -> DPO / Preference Optimization：SFT 定后“搜索起点”

Base Model -> SFT Policy -> Preference Pairs / DPO -> Reward/RL -> Final Policy

SFT 太弱	SFT 太强/太窄
不会稳定遵循格式，preference 数据噪声变大	policy 多样性降低，candidate 差异不足
tool schema 常错，后续 reward 浪费在语法问题	过度模仿 teacher，限制探索空间
reasoning 可读性差，judge/verifier 成本高	特定风格成为难以纠正的 prior

Know-Why： SFT 的目标不是把模型做到“最终最优”，而是进入一个 downstream optimization 可工作的行为区域：合法、稳定、可评、仍保有足够能力与多样性。

25. SFT -> RL / RLVR : Cold Start 与 Behavior Prior

RL 前 SFT 作用	具体意义
Format Bootstrap	先学会可解析 answer/tool format，reward 才有效
Readability	避免 RL 找到难读但高 reward 的策略
Task Prior	把 policy 放到有成功概率的区域
Safety / Policy Anchor	减少 RL 探索进入不可接受行为
Verifier Compatibility	输出可被 judge/test/parser 稳定评估

公开研究证据：DeepSeek-R1-Zero 展示了无 preliminary SFT 的纯 RL 可以涌现强 reasoning，但存在 poor readability 和 language mixing；DeepSeek-R1 因而加入 cold-start data 和多阶段训练。[7]

工程推论：在可验证 RL 中，SFT 的核心价值之一是提高初始 non-zero reward rate。如果 rollout 几乎全部失败，RL 获得的有效梯度和搜索效率都很差。

26. Distillation SFT：把 Search 发现的行为压进 Student

Strong/Reasoning Teacher -> generate N trajectories -> verify/filter -> SFT Student -> optional preference/RL

设计点	建议
Teacher diversity	不要只用单一 temperature/单一 teacher 风格
Verifier	优先保留可验证正确 trajectory
Length control	避免学生学会无谓超长 reasoning
Task mix	teacher 强项不能覆盖产品全部目标
Student capacity	小模型可能无法完整模仿长轨迹，应压缩/分级

公开研究证据：DeepSeek-R1 发布了多个基于 Qwen/Llama 的 distilled dense models，体现了“强 reasoning policy -> 生成轨迹 -> SFT/蒸馏到更小模型”的现代 post-training 组合。[7]

27. SFT Failure-Driven Flywheel：从 Eval Failure 反推新的 Demonstration

Production SFT is an iterative control loop

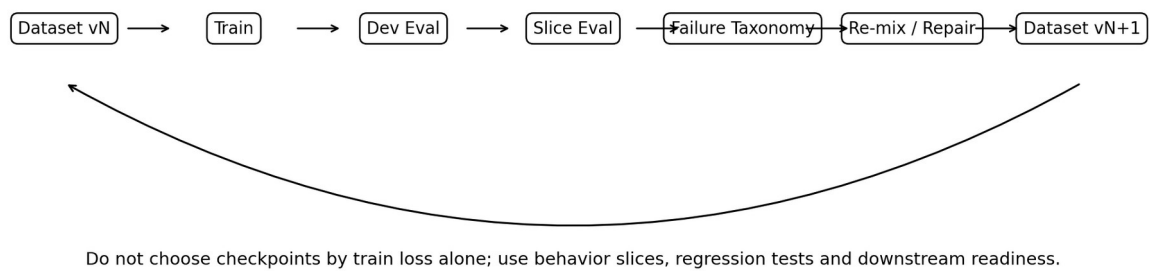


图 27-1 SFT 数据应该随模型失败分布动态演化。

Eval failure -> classify root cause -> collect/generate expert demonstration -> verify -> add to mixture -> retrain -> regression eval

Failure	不要只做	更好的数据动作
JSON 格式错	加更多通用 QA	加入对照的格式约束示范 + parser filter
Tool 参数错	加更多 tool calls	加入相似 tool 的 hard disambiguation examples
不会恢复	只加入成功轨迹	加入 failed observation -> recovery action
回答太长	简单截短所有数据	构造长度可控、同任务不同 verbosity 的示范
领域遗忘	继续加 domain data	增加 general anchor + 非目标域 regression

28. Production SFT Contract：训练开始前必须冻结的契约

```
sft_run:
  base_model: model@sha256
  tokenizer: tokenizer@sha256
  chat_template: chat_template_v7
  dataset: sft_mix_v12
  target_policy:
    assistant_only_loss: true
    include_tool_calls: true
    include_reasoning: selective
  sequence:
    max_length: 8192
    packing: true
    truncation_policy: preserve_target
  optimization:
    method: full_ft
    lr: <sweep>
    token_batch: <fixed>
    warmup: <ablate>
  eval:
    dev_slices: [general_if, code, math, tool, multilingual, safety]
    regression_slices: [base_knowledge, long_context]
  release_gate:
    no_major_regression: true
    format_validity: ">= target"
    downstream_ready_for_dpo_rl: true
```

为什么叫 Contract：因为 template、mask、dataset、tokenizer、optimizer 和 eval 任意一个变化，都可能让“同一个 SFT 实验”变成不同训练目标。实验平台必须把这些作为版本化输入，而不是 notebook 隐式状态。

29. 五套可复用 SFT Recipe

Recipe	数据	Mask	训练关注点	下一阶段
General Assistant	高质量多任务对话	assistant-only	diversity / style / IF	DPO
Reasoning Cold Start	verified reasoning + answer	reasoning+answer 或 selective	correctness / readability	RLVR
Tool Agent Bootstrap	tool call + result + final	assistant action spans	schema / tool choice / recovery	online rollout/RL
Domain Adapter	domain expert + general anchors	assistant-only	forgetting / terminology	domain eval/DPO
Distilled Student	teacher verified trajectories	target spans	capacity / length / teacher bias	optional DPO/RL

默认原则：先用小规模 sweep 找稳定 LR/batch/epoch，再扩数据；先验证 serialization/mask，再调优化器；先有 slice eval，再进行昂贵 full run。

30. SFT 的 18 类常见失败模式

#	失败模式	根因	修复方向
1	Train/Inference template mismatch	serialization 不一致	模板版本化 + golden decode
2	Loss mask 错	target span 错位	batch mask 可视化
3	只看 loss	NLL 与行为错位	slice eval + generation audit
4	数据越多越好	低质量/冗余稀释信号	quality/diversity/value selection
5	只有单轮 QA	interaction coverage 不足	multi-turn / tool / constraint
6	同一 teacher 风格	synthetic monoculture	多 teacher/source/diversity
7	Reasoning CoT 未验证	错误轨迹写入 policy	answer/test/verifier
8	Tool trace 只有成功	不会 recovery	加入失败->恢复示范
9	领域 SFT 训窄	无 general anchor	replay/mix/PEFT
10	LR 过大	policy 漂移	保守 LR + update norm
11	Epoch 太多	小数据重复过拟合	early stop / data expand
12	Packing 串样本	边界/mask bug	attention isolation tests
13	答案尾部被截断	truncation 只保 prompt	preserve-target truncation
14	PEFT 默认 rank	容量未验证	rank/modules ablation
15	Benchmark contamination	训练集泄漏	decontamination
16	安全样本过度	over-refusal	policy balance + benign slice
17	忽视非目标回归	catastrophic forgetting	regression suite
18	SFT 追求最终最优	压制后续探索	以 downstream-ready 为 release gate

31. SFT 成熟度模型与上线检查单

Level	系统特征	典型能力
L0 - Fine-tune Script	text -> trainer.train()	可训练但不可解释
L1 - Reproducible SFT	dataset/tokenizer/template/version 固化	可复现
L2 - Behavior-Aware	assistant mask、mixture、slice eval	知道梯度在学什么
L3 - Stage-Aware	reasoning/tool/agent/domain recipe 分流	为 DPO/RL 提供正确先验
L4 - Adaptive SFT Factory	failure mining、dynamic remix、regression controller	持续按模型缺口制造示范

上线前检查	Pass 条件
Template parity	训练与部署 tokenizer/chat template 完全一致
Mask audit	随机样本逐 token 核验 target span
Data lineage	每个 source / generator / verifier 可追溯
Contamination	核心 benchmark 与产品 holdout 已隔离
Behavior eval	目标能力达标
Regression eval	通用/多语言/安全/长上下文无不可接受回归
Checkpoint	不是仅按最低 val loss 选择
Downstream readiness	DPO/RL 需要的合法格式、成功率和多样性已满足

最终 Know-Why: SFT 是后训练中成本相对可控、反馈最直接的一层，但它会强烈塑造后续优化的可达空间。高质量 SFT 的价值不只是“先把模型变好”，更是“让后续 Preference/RL 在正确的行为流形上工作”。

32. 核心考料推序

来源	为何值得读
[1] Ouyang et al., 2022	Training language models to follow instructions with human feedback. arXiv:2203.02155. SFT -> RM -> PPO 的经典起点。
[2] Chung et al., 2022	Scaling Instruction-Finetuned Language Models. arXiv:2210.11416. 任务规模、模型规模与 CoT instruction tuning。
[3] Zhou et al., 2023	LIMA: Less Is More for Alignment. arXiv:2305.11206. 质量/多样性、小数据 SFT 与 perplexity/checkpoint 讨论。
[4] Grattafiori et al., 2024	The Llama 3 Herd of Models. arXiv:2407.21783. 现代大模型 post-training、SFT/RS/偏好与 tool/reasoning 能力。
[5] Lambert et al., 2024	Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. 开放 SFT/DPO/RLVR recipe 与评测。
[6] Pareja et al., 2024	Unveiling the Secret Recipe: A Guide For Supervised Fine-Tuning Small LLMs. arXiv:2412.13337. 系统超参研究。
[7] DeepSeek-AI, 2025	DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. cold-start SFT、RL、distillation。
[8] Hu et al., 2021	LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
[9] Dettmers et al., 2023	QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314.
[10] Jain et al., 2023	NEFTune: Noisy Embeddings Improve Instruction Finetuning. arXiv:2310.05914.
[11] Luo et al., 2023	An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning. arXiv:2308.08747.
[12] Hugging Face TRL, accessed 2026-08-13	SFTTrainer documentation: chat templates, assistant_only_loss, completion_only_loss, packing, padding-free 等工程契约。
[13] Xu et al., 2023	WizardLM: Empowering Large Language Models to Follow Complex Instructions. arXiv:2304.12244.
[14] Dong et al., 2024	Self-play with Execution Feedback / AutoIF. arXiv:2406.13542.
[15] Wang et al., 2022	Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. arXiv:2204.07705.
[16] Wang et al., 2022	Self-Instruct: Aligning Language Models with Self-Generated Instructions. arXiv:2212.10560.

推荐顺序：先读 [1] 建立 SFT 在后训练链的位置；再读 [3] 理解质量/多样性与“小数据”；读 [4][5] 看现代 recipe；读 [6] 看超参实证；研究 reasoning/agent 再读 [7][14]；资源受限训练看 [8][9][12]。

研究说明：本手册将公开论文/官方文档中的实证结论与工程推论区分呈现。不同模型规模、数据分布、tokenizer、训练实现和评测体系会显著改变最优 SFT recipe；表中的建议应通过目标模型上的 ablation 与回归评测确认。